RSCL — Algorithm 18: Offline Transition-Clearance Analysis for Selecting Safe Shape Orders

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)

Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/acceptance/strategist.py`	`sha256 bd7547604ba1`	2026-11-12 01:19 UTC	modified vs 983eede
`hu-telepresence-gazebo/festival/formation_planner.py`	`sha256 f2a21e7476cb`	2026-11-12 00:52 UTC	modified vs 10ee643
`hu-telepresence-gazebo/acceptance/formation_show.sh`	`sha256 c3504bbd4843`	2026-11-12 01:20 UTC	modified vs 983eede
`hu-telepresence-gazebo/acceptance/results/formation_transitions.json`	`sha256 810f8a87a78c`	2026-11-12 00:54 UTC	clean

1. Purpose and context

A formation show chains transitions; some shape pairs force aircraft to cross closely (for example line → diamond plans a 0.956 m pass, close to the 0.82 m watchdog that would freeze an aircraft mid-show). Before the show, an offline analysis computes a clearance figure for each ordered pair of shapes and the fixed show orders are chosen among transitions with large clearance. The per-pair computation is exactly `strategist.plan()` (Algorithm 14.1) applied with the fleet placed on the slots of the preceding shape. The 104-shape table is stored in `acceptance/results/formation_transitions.json` ("offline formation_planner.assign from slots(a,5) at ALT to slots(b,5); 61-sample straight-line clearance (same as strategist.plan)"; its metadata field `veto_m` is 1.0 m, the value in force when the table was generated). The live Safety Officer veto in `strategist.py` has since been raised to `VETO_M` = **1.10 m**, with the code comment that live flights came up to about 0.15 m closer than planned (line→diamond 0.96 m planned / 0.81 m flown).

2. Inputs, outputs

- **Input:** set of shape keys K (23 flown shapes originally; 104 in the current table); planner constants (MIN_SEP 0.88 m, ALT 1.90 m, BOUND 3.0 m).
- **Output:** matrix C[a→b] = tightest planned pass in metres, or null if ASSIGN finds no separation-safe assignment; a show order $s_1..s_m$ with $\min_i C[s_i \to s_{i+1}]$ above the target margin.

3. Algorithm

Algorithm 18.1 Clearance matrix and order check

Source: acceptance/strategist.py: plan; festival/formation_planner.py: slots, assign

```
procedure CLEARANCE-MATRIX(K)
  for a in K do
    fleet ← { name_i : (SLOTS(a,5)_i , z = 1.90) for i = 1..5 }      // fleet order onto a's slot order
    for b in K, b ≠ a do
      try p ← PLAN(b, fleet)                                          // Alg. 14.1: ASSIGN + 61-sample min pairwise distance
          C[a→b] ← p.clear
      except no-assignment: C[a→b] ← null
  return C
function ORDER-MIN(s_1..s_m, C)
  return min_i C[s_i → s_{i+1}]                                       // null counts as infeasible
// selection: candidate orders were composed by hand from transitions with high C;
```

```
12 // the acceptance target written in formation_show.sh is ≥ 1.23 m (default) / ≥ 1.10 m, minimum 1.109 m
   (SHAPES20)
```

3.1 Flowchart and data flow

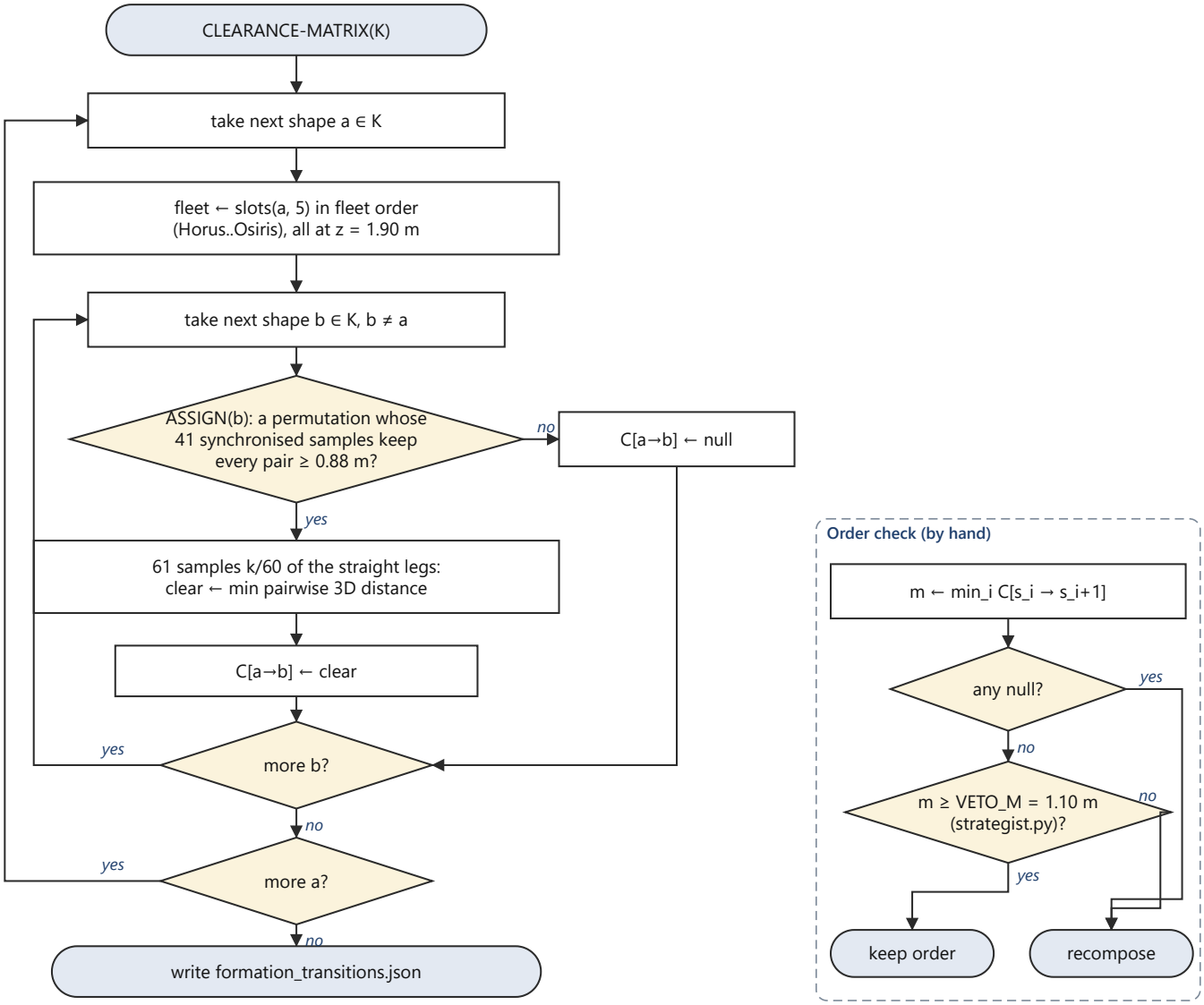


Figure 18.1 Flowchart of CLEARANCE-MATRIX (left) and of the order check (right). ASSIGN is Algorithm 5 (`formation_planner.assign`), the 61-sample clearance is `strategist.plan`. The order check is applied by hand; no code searches over orders.

The analysis is stateless: the matrix is a pure function of the shape set and planner constants, and nothing is carried between pairs. Instead of a state machine, Figure 18.2 shows where the offline figures and the live checks enter the show.

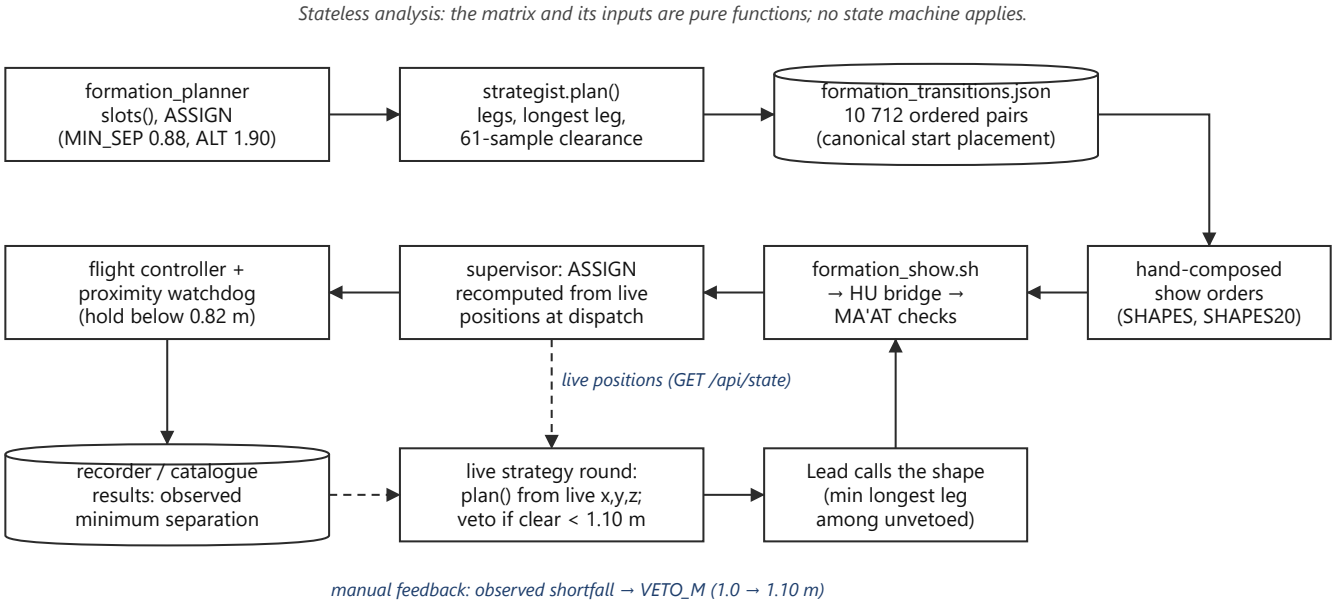


Figure 18.2 Data flow. Solid arrows: data used automatically; dashed: live positions and the manual feedback by which observed shortfalls led to raising VETO_M.

Note on chaining: the matrix starts each transition from the *canonical* fleet-order placement on shape a. In a flown chain the aircraft occupy shape a in the permutation chosen by the previous ASSIGN. For the two show sequences below, a chained recomputation (start from fleet-order circle, then carry the assigned permutation forward) gives the same values as the matrix, but this equality is not guaranteed in general.

4. Parameters

Parameter	Value
Path samples	61 (k/60), synchronised straight lines, all at ALT = 1.90 m
Assignment	ASSIGN of Algorithm 5 (41 samples at 0.88 m as feasibility filter; min sum of squared legs)
Veto reference	`VETO_M` = 1.10 m in the live strategy round (was 1.0 m; the stored table still records 1.0); supervisor watchdog hold 0.82 m

5. Complexity

$|K|(|K|-1)$ calls of PLAN, each $O(5!\cdot 41\cdot 10 + 61\cdot 10)$: about $5\cdot 10^4$ distance evaluations per pair; for $|K| = 104$, 10 712 pairs $\approx 5\cdot 10^8$ evaluations (minutes in pure Python, offline).

6. Measured results (offline recomputation and live evidence)

Quantity	23 original shapes	104 shapes (current table)
Ordered pairs	506	10 712
Solvable by ASSIGN	494	10 438
Tightest pass ≥ 1.0 m	456	8 986
Tightest pass ≥ 1.10 m (current veto)	–	5 955
Minimum / median / maximum	0.882 / 1.162 m / –	0.880 / 1.127 / 2.302 m

The 23-shape figures were recomputed independently here with `strategist.plan` and agree with the stored table. Show sequences (consecutive transitions, m):

- **Default 11-shape sequence:** circle→zigzag 1.773, zigzag→square 1.812, square→diamond 1.491, diamond→cross 1.450, cross→X 1.355, X→triangle 1.231, triangle→circle 1.500, circle→V 1.233, V→wedge 1.372, wedge→circle 1.233. Minimum **1.231 m** (X→triangle): consistent with the script's "≥ 1.23 m" comment.
- **SHAPES20 sequence:** minimum **1.109 m** (L→circle, the closing return), then wedge→L 1.156 m and two rows→right echelon 1.162 m. The script comment now reads "every transition ≥ 1.10 m planned clearance (computed offline; minimum 1.109 m, L → circle)", which agrees with this recomputation. The margin over the raised veto is only 9 mm: a strategist veto check on the same canonical placement would accept every SHAPES20 transition, but L→circle only just.
- Line → diamond: 0.956 m (vetoed; used as the counter-example in the script and in `test_strategist.py`).

Live comparison. In the earlier 25-shape catalogue flight the observed minimum separations were 0.906–1.662 m with zero watchdog holds (Algorithm 5). Tonight's live validation of the extended catalogue gives the first per-transition evidence on how far a flown pass can fall below the plan; it was flown as one chain from the live positions (it does not use the canonical starts of the matrix):

Run (acceptance/results/)	Outcome	Planned at dispatch vs observed minimum
`catalog_new/` (01:18:49 UTC)	92 entries: 44 VALIDATED (0 holds), 1 EXPERIMENTAL, 47 REFUSED by the supervisor ("No separation-safe straight-line assignment found")	VALIDATED: planned ≥ 1.15 m, observed ≥ 1.098 m, largest shortfall 0.181 m (v_right 1.279 → 1.098 m)
same run, pentomino_f → pentomino_w	EXPERIMENTAL: proximity watchdog engaged (`proximity_holds` = 3 recorded), shape error 0.55 m	0.884 m planned → 0.633 m observed (Ra–Anubis); the matrix value for this pair from the canonical placement is **1.25 m**
`catalog_new2/` (01:20:39 UTC)	first step line from the unsettled pentomino_w configuration: EXPERIMENTAL, 3 holds; the remaining 56 refused by the supervisor	0.894 m planned → 0.551 m observed; the matrix has **null** for pentomino_w → line

Two conclusions follow from these records. (i) The matrix value is not a bound for a flown chain: for pentomino_f→pentomino_w it predicted 1.25 m from the canonical placement, while the plan from the actual positions was 0.884 m. (ii) The flown pass fell 0.25–0.34 m below the plan in the two tight cases, more than the 0.15 m quoted in the strategist comment; with VETO_M = 1.10 m and the largest shortfall seen in a validated transition (0.181 m) the expected flown minimum stays above the 0.82 m watchdog, but the two tight cases show that a sub-veto plan (0.88–0.90 m) does trigger holds. Note that the catalogue runs do not go through the strategist veto; they test the supervisor's own planner (MIN_SEP 0.88 m).

7. Properties and limitations

- Guaranteed: the matrix is deterministic and computed with the same planner code as the supervisor; a null entry means the supervisor would refuse that transition from the canonical placement.
- Not guaranteed: the figure is a planned straight-line clearance, not a flown one (controller saturation, Algorithm 5); it depends on the starting permutation; the supervisor recomputes the assignment from live positions at dispatch, which may differ.
- Order selection itself is not automated in the repository (no search code); orders were composed by hand and checked with this analysis.
- The matrix assumes all aircraft start at ALT = 1.90 m. With `LEVELS=1` the show moves each aircraft to a random height in 1.1–2.7 m (or a 3-D height profile) after each shape, so the next transition starts from different heights. For the same assignment this can only increase the sampled 3-D distances (horizontal motion is unchanged, a vertical component is added), but ASSIGN may then choose a different permutation, so the matrix value is not a bound in that case either. The live strategist `plan()` uses the live z and is not affected.
- Sampling resolution: for straight synchronised legs the relative position of a pair is linear in the path fraction, so the true minimum can lie between two of the 61 samples; the sampled value overestimates it by at most

|d|/120, where |d| is the length of the pair's relative displacement (e.g. ≤ 0.05 m for |d| = 6 m).
- The Lead's evidence record still states the rule as "tightest pass ≥ 1.0 m" while the veto applied is 1.10 m (text only; the decision uses `VETO_M`).

8. Comparison with alternatives and related work

Choosing which robot goes to which slot and checking that the simultaneous motions stay separated is a studied problem in multi-robot planning. The table places the implemented method (exhaustive assignment over 5! permutations with sampled straight-line checks, plus an offline matrix) among established approaches.

Approach	Idea	Guarantee	Relation to ours
CAPT, Turpin, Michael & Kumar [1]	Assign goals by minimising the sum of squared distances, fly synchronised straight lines	Proven collision-free in obstacle-free space when start and goal positions satisfy a minimum spacing condition (centralised c-CAPT)	ASSIGN uses the same cost and the same synchronised straight-line parametrisation, but rejects permutations by sampling instead of relying on the theorem, and adds a separation margin. We have not checked whether our slot spacings satisfy CAPT's condition.
Conflict-Based Search, Sharon et al. [2]	Two-level search: high-level conflict tree, low-level single-agent paths; optimal multi-agent path finding	Optimal and complete on discrete graphs	Plans arbitrary paths around conflicts; ours only accepts or rejects straight lines and has no fallback path (the supervisor refuses instead).
Quadrotor swarm planning, Hönig et al. [3]	Roadmap, discrete planning, continuous trajectory refinement; models downwash with ellipsoidal robot shapes	Collision-free trajectories in obstacle-rich environments, dense formations	Ours treats aircraft as points with a spherical margin and ignores downwash; adequate only because the simulation is used.
Continuous collision detection, e.g. FCL [4]	Exact first time of contact between moving geometric objects	No missed contacts between samples	Ours samples 61 (planner: 41) instants; the error bound is given in Section 7.

Honest assessment. The method is simple, deterministic and runs the same code as the supervisor, which makes the matrix easy to reproduce and to audit. It is weaker than the alternatives in every guarantee: it has no completeness (a transition the straight-line planner refuses has no alternative path), no proof of separation (sampled, point robots, no downwash or controller dynamics), and the offline matrix assumes a canonical starting permutation and altitude, which tonight's live run showed overstated the dispatch-time plan by 0.37 m and the flown minimum by 0.62 m (pentomino_f→pentomino_w). The protection in flight remains the supervisor's live recomputation and the 0.82 m watchdog, not this analysis.

References

1. M. Turpin, N. Michael, V. Kumar, "CAPT: Concurrent assignment and planning of trajectories for multiple robots," *The International Journal of Robotics Research*, 33(1):98–112, 2014. doi:10.1177/0278364913515307. https://ri.cmu.edu/publications/capt-concurrent-assignment-and-planning-of-trajectories-for-multiple-robots
2. G. Sharon, R. Stern, A. Felner, N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, 219:40–66, 2015. https://people.engr.tamu.edu/guni/pistar/Papers/CBS-AIJ15.pdf
3. W. Hönig, J. A. Preiss, T. K. S. Kumar, G. S. Sukhatme, N. Ayanian, "Trajectory Planning for Quadrotor Swarms," *IEEE Transactions on Robotics*, 34(4):856–869, 2018. https://act.usc.edu/publications/Hoenig_TRO2018.pdf
4. J. Pan, S. Chitta, D. Manocha, "FCL: A General Purpose Library for Collision and Proximity Queries," *IEEE International Conference on Robotics and Automation (ICRA)*, 2012. https://gamma.cs.unc.edu/FCL/fcl.pdf

9. Tests and relation to other algorithms

`acceptance/tests/test_strategist.py::test_tight_transition_is_below_veto`; `test_formation_catalog_offline.py` (every shape builds five safe slots; new shapes clear for subsets). Uses Algorithms 5 and 14.1; its results fix the sequences of Algorithm 16.

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.